



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського» Факультет інформатики та обчислювальної
техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Компоненти програмної інженерії»
Лабораторна робота №3
"Мікросервісна архітектура"

Виконав:

Студент II курсу

гр. ІП-33

Соколов О. В.

Перевірив:

Зарічковий О. А.

Лабораторні робота №3.

Мікросервісна архітектура

Тема: Мікросервісна архітектура.

Мета: Застосувати мікросервісну архітектуру для проектування складних систем.

Постановка задачі:

1. Між-сервісна взаємодія:
 - 1.1 Визначити які сервіси між собою спілкуються синхронно.
 - 1.2 Визначити які сервіси між собою спілкуються асинхронно, за допомогою яких систем (kafka/rabbitmq/...)
2. Робота з даними.
 - 2.1. Визначити, який патерн використовується для роботи з базою даних (db per service or shared db)
 - 2.2. Визначити, як нагачуються міграційні скрипти для бази даних.
 - 2.3. Визначити, як саме відбувається синхронізація даних між сервісами.
3. Оновити діаграми, зобразити всі зв'язки між сервісами.
4. Вказати, які саме патерни для мікросервісної архітектури використовуються.

Виконання

1. Міжсервісна взаємодія

1.1. Синхронне спілкування

Синхронне спілкування використовується для операцій, що вимагають негайної відповіді від сервісу, та реалізується за допомогою REST або gRPC викликів через API Gateway (або безпосередньо, якщо це доцільно).

a) Клієнт → API Gateway:

Запити на автентифікацію, пошук контенту, отримання текстів пісень, обробка платежів.

Наприклад, користувач запитує пошук музики, який проходить через Search Service.

b) Users Service ↔ Playlists Service:

При створенні/редагуванні плейлистів перевіряється, чи має користувач відповідні права, отримання даних з профілю здійснюється синхронно.

c) Payments Service → Users Service:

Після оплати синхронне оновлення статусу облікового запису (наприклад, підписка преміум) для негайного застосування змін.

d) Streaming Service → Lyrics Service:

При відтворенні треку API звертається за синхронним запитом для отримання синхронізованих текстів пісень.

e) API Gateway → Сервіси (Search, Users, Payments тощо):

Безпосередні REST або gRPC виклики для отримання даних, обробки запитів клієнтів.

1.2. Асинхронне спілкування

Асинхронне спілкування використовується для обробки подій, що не потребують негайної відповіді, для забезпечення високої масштабованості та розвантаження системи. В основному реалізується за допомогою брокерів повідомлень (Kafka, RabbitMQ).

a) Streaming Service → Analytics Service:

Публікація подій відтворення (старт, пауза, пропуск) у брокер повідомлень. Analytics Service підписується на ці події для обчислення статистики та аналізу поведінки користувачів.

b) Streaming Service → Recommendations Service:

Події відтворення, лайки/дизлайки та інші взаємодії публікуються до брокера (наприклад, Kafka). Recommendations Service отримує ці дані для персоналізації рекомендацій.

c) Users Service → Emails Service:

Події, наприклад, успішна реєстрація, оновлення профілю або скидання пароля, публікуються в брокер повідомлень. Emails Service, який підписаний на відповідні канали, запускає процес надсилання листів.

d) Playlists Service → Analytics & Recommendations:

При створенні, редагуванні або видаленні плейлистів публікуються події, щоб трекати взаємодії та коригувати алгоритми рекомендацій.

e) Payments Service → Analytics & Emails Service:

Події оплати публікуються для аналітики та надсилання повідомлень (наприклад, сповіщення про успішну оплату).

2. Робота з даними

2.1. Використання баз даних

Паттерн DB per Service. Кожен мікросервіс володіє власною базою даних. Це дозволяє ізолювати дані, підвищити масштабованість та забезпечити незалежне оновлення сервісів. Це покращує ізоляцію сервісів, дає можливість оптимізувати вибір технологій БД відповідно до потреб конкретного сервісу.

2.2. Міграції

Використання інструментів на зразок **Flyway** або **Liquibase** для автоматичного виконання міграційних скриптів під час розгортання або оновлення сервісу. Скрипти зберігаються в репозиторії відповідного сервісу й автоматично виконуються в CI/CD пайплайні.

2.3. Синхронізація даних

Методи синхронізації:

a) Event Sourcing / Change Data Capture (CDC). Зміни даних в одному сервісі публікуються як події через брокер повідомлень. Інші сервіси підписуються на ці події, отримуючи актуальні оновлення.

б) Синхронні API-запити. При необхідності отримання актуальної інформації один сервіс може зробити запит до іншого сервісу.

с) Кешування даних. Використання Redis або подібних систем для зменшення навантаження на сервіси, що роблять синхронні запити.

3. Діаграма

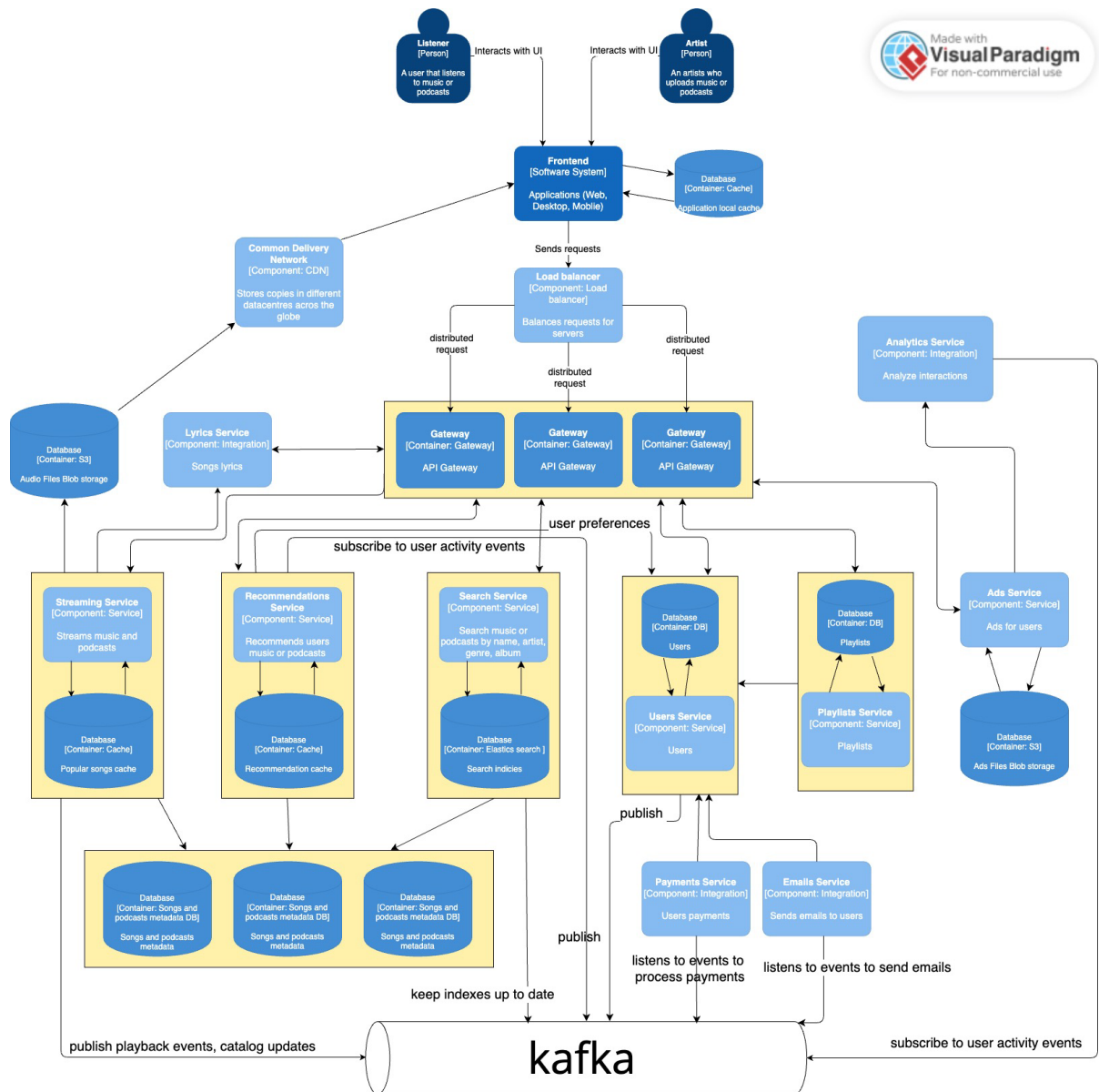


Рис. 1 – Оновлена діаграма

3.1 Елементи оновленої діаграми

1. **Клієнтський шар:**
 - Мобільні додатки, веб-браузери.
2. **Рівень мережевого входу:**
 - **Load Balancer:** Розподіл вхідного трафіку між API Gateway або безпосередньо між екземплярами сервісів.
 - **CDN:** Кешування та доставка статичного контенту (аудіо, зображення).
3. **API Gateway:**
 - Централізований вхід для синхронних запитів клієнтів, що направляє запити до відповідних мікросервісів.
 - Може виконувати автентифікацію, авторизацію, фільтрацію запитів.
4. **Сервісний шар:**
 - **Streaming Service:** Комунікація із CDN для доставки контенту, публікація подій (до Analytics та Recommendations).
 - **Recommendations Service:** Підписка на асинхронні події від Streaming, Users, Playlists; синхронні запити для отримання даних про користувача.
 - **Search Service:** Синхронна взаємодія з клієнтами, асинхронне оновлення індексу за подіями від Streaming та Playlists.
 - **Users Service:** Синхронне обслуговування запитів реєстрації, входу, оновлення профілю; публікація подій для Emails та Analytics.
 - **Playlists Service:** Синхронне управління плейлистами (CRUD операції), перевірка користувацьких прав через Users Service; публікація подій.
 - **Ads Service:** Синхронне визначення контенту для показу реклами, можлива асинхронна адаптація на основі поведінкових сигналів.
 - **Analytics Service:** Підписка на події від Streaming, Users, Playlists, Payments; надання даних для звітності через синхронні API.
 - **Emails Service:** Асинхронна обробка подій від Users та Payments для надсилання повідомлень.
 - **Payments Service:** Синхронна обробка платіжних операцій, оновлення стану через Users Service; публікація подій для Analytics і Emails.
 - **Lyrics Service:** Синхронна взаємодія при запиті текстів пісень від Streaming Service або клієнтського додатку.
5. **Брокер повідомлень (Kafka / RabbitMQ):**

- Централізований вузол для асинхронного обміну повідомленнями між сервісами (наприклад, між Streaming, Analytics, Recommendations, Users, Playlists, Payments та Emails).

6. **Бази даних для кожного сервісу:**

- Кожний сервіс має власну БД із зазначенням патерну **DB per Service** та інструментів автоматичних міграцій (Flyway/Liquibase).

4. Паттерни мікросервісної архітектури

a) **API Gateway** – Єдиний вхідний пункт для клієнтських запитів із додатковими можливостями аутентифікації, авторизації, маршрутизації та обмеження швидкості запитів.

b) **DB per Service** – Кожен мікросервіс має власну базу даних, що забезпечує ізоляцію даних і дозволяє незалежне масштабування.

c) **Event-Driven Architecture** – Використання брокера повідомлень (Kafka, RabbitMQ) для асинхронної взаємодії між сервісами (публікація/підписка на події).

d) **Circuit Breaker Pattern** – Для синхронних викликів між сервісами, що забезпечує захист у разі збою (наприклад, між API Gateway та конкретними сервісами).

Висновок:

У результаті виконання практичної роботи було розроблено мікросервісну архітектуру для стрімінгового сервісу музики та подкастів. Було визначено міжсервісну взаємодію: частина сервісів спілкується синхронно (наприклад, search, users, playlists, lyrics, recommendations через API Gateway), тоді як для обміну подіями між сервісами (analytics, emails, ads, recommendations) використовується асинхронна взаємодія через брокери повідомлень (Kafka або RabbitMQ). Для зберігання даних застосовано підхід "**DB per Service**", що забезпечує незалежність сервісів та полегшує масштабування. Міграції для кожної бази даних здійснюються через автоматизовані інструменти, наприклад, Flyway або Liquibase, в рамках CI/CD процесу. Синхронізація даних між сервісами реалізується через подієво-орієнтовану архітектуру, де сервіси реагують на події, не порушуючи ізоляції. У діаграму архітектури були додані зв'язки між сервісами, типи комунікації та відповідні патерни, такі як API Gateway, Event-Driven Communication, Database per Service, CQRS та Service Discovery. Таким чином, розроблена архітектура є масштабованою, гнучкою та відповідає сучасним вимогам до побудови розподілених систем.

